

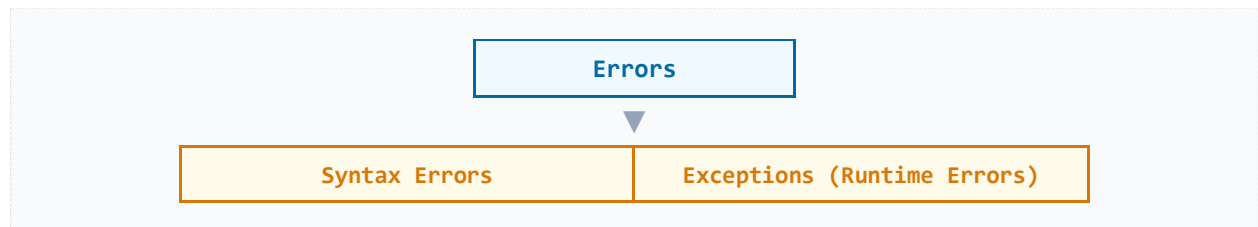
Understanding Errors & Exceptions: Handling the Unexpected

A practical guide to the difference between syntax errors and runtime exceptions — covering common exception types, try/except handling, catching specific and general errors, and raising both built-in and custom exceptions.

Syntax Errors	Runtime Exceptions
try / except	Common Exception Types
raise Keyword	Custom Exceptions

1. What Is an Error?

An error is a problem in a program that prevents it from running correctly. Python errors fall into two main categories: syntax errors, which occur before execution, and exceptions (runtime errors), which occur during execution. Exceptions are, in fact, a type of error.



2. Syntax Errors

A syntax error occurs when Python cannot understand the code because it breaks Python's grammar rules.

syntax_error_1.py

```
if 5 > 3
    print("Hello")
```

OUTPUT

SyntaxError: expected ':'

Python stops before running the program at all.

syntax_error_2.py

```
print("Hello"
```

OUTPUT

SyntaxError: '(' was never closed

Why? Python first reads the entire program (parses it). If the grammar is wrong anywhere, execution never starts at all.

3. Exceptions

An exception happens while the program is running. The syntax is correct, but something unexpected occurs.

exception_example.py

```
a = 10
b = 0

print(a / b)
```

OUTPUT

ZeroDivisionError: division by zero

Python successfully started execution. It executed `a = 10` and `b = 0` without any issue. Then, when it reached `a / b`, it discovered a problem and raised an exception.

3.1 Everyday Analogy

Imagine sitting an exam.

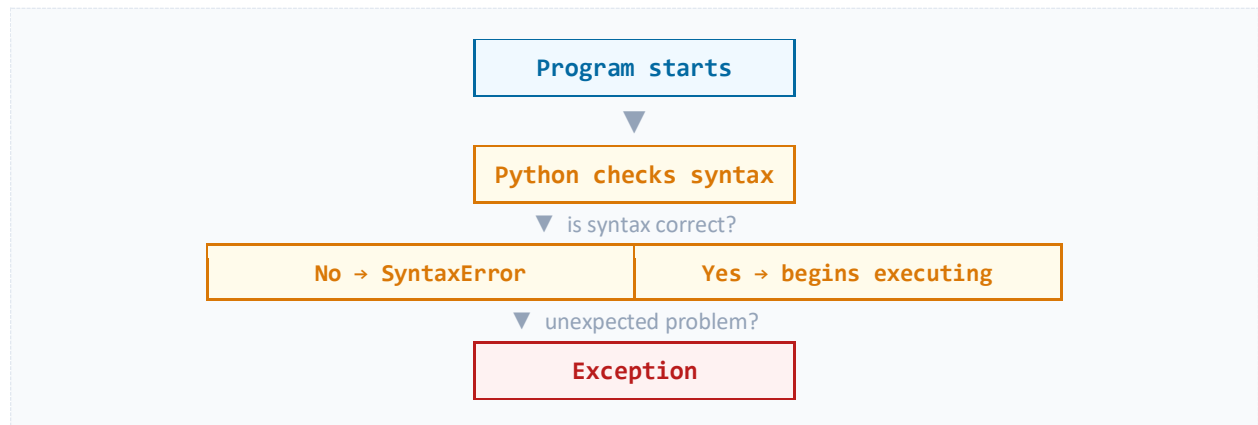
Syntax Error

Exception

Syntax Error — the roll number is written incorrectly. The examiner refuses to accept the paper; the exam never even starts.

Exception — the paper is accepted, and halfway through the exam the pen runs out of ink. The exam had already started, but something unexpected happened.

3.2 Visual Timeline



4. Common Exceptions

Python defines many built-in exception types. Each signals a different kind of runtime problem.

Exception	Triggering Code	Meaning
ZeroDivisionError	<code>print(10 / 0)</code>	Division by zero
NameError	<code>print(age)</code>	Name 'age' is not defined
TypeError	<code>print(5 + "10")</code>	Mismatched operand types
ValueError	<code>int("hello")</code>	Value can't convert to the target type
IndexError	<code>numbers[5]</code>	List index out of range
KeyError	<code>student["age"]</code>	Dictionary key does not exist
FileNotFoundError	<code>open("abc.txt")</code>	File does not exist on disk
AttributeError	<code>x.append(5)</code>	Object has no such method/attribute

5. How Python Handles Exceptions

Without any handling, an exception stops the program immediately:

unhandled.py

```
print("Start")
print(10 / 0)
print("End")
```

OUTPUT

```
Start
ZeroDivisionError
```

The last line never executes because the exception stops the program before it gets there.

5.1 Handling Exceptions

Python provides `try` and `except` so the program can continue instead of crashing.

try_except.py

```
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Cannot divide by zero")

print("Program continues")
```

OUTPUT

```
Cannot divide by zero
Program continues
```

5.2 Multiple Exceptions

multiple_except.py

```
try:
    num = int(input())
    print(10 / num)

except ValueError:
    print("Enter a valid number.")

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

5.3 Catch Any Exception

catch_any.py

```
try:
    x = int("abc")
except Exception as e:
    print(e)
```

OUTPUT

```
invalid literal for int() with base 10: 'abc'
```

`e` stores the exception object, which contains details about what went wrong.

6. Raising Exceptions

6.1 Raising a Built-In Exception

An exception can be raised intentionally, whenever code detects an invalid situation.

`raise_builtin.py`

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative.")
```

OUTPUT

ValueError: Age cannot be negative.

6.2 Creating a Custom Exception

`custom_exception.py`

```
class NegativeAgeError(Exception):
    pass

age = -3

if age < 0:
    raise NegativeAgeError("Negative age is not allowed.")
```

Custom exception classes let a program signal domain-specific problems using a name that clearly communicates intent, while still behaving like any other Python exception — including being catchable with `try / except`.

7. Error vs. Exception

Feature	Error (Syntax Error)	Exception
When it occurs	Before execution	During execution
Can the program start?	No	Yes
Cause	Invalid Python syntax	Unexpected runtime condition
Can it be handled with <code>try/except</code> ?	No	Yes
Example	Missing :	Divide by zero

7.1 Summary

- Syntax Error — the code breaks Python's grammar; the program never starts.
- Exception — the code is syntactically correct, but an unexpected problem occurs while it is running.
- `try / except` is used to catch and handle exceptions so the program can continue.
- Built-in or custom exceptions can be raised whenever code detects an invalid situation.

A common misconception, corrected: people often talk about “error vs. exception” as if they were two separate, unrelated categories. In Python, an exception is itself a kind of error that occurs at runtime. A more accurate comparison is syntax errors versus exceptions (runtime errors) — two categories under the same umbrella, not two competing concepts.

